COMPUTING 2 - PARALLEL COMPUTING

Performance models - decomposition - collectives - CPU parallelism - shared-memory correctness

1. PERFORMANCE EQUATIONS

Speedup: $S(p) = T_1 / T_p$ | Efficiency: $E = S(p) / p$

Parallel overhead: $T_o = pT_p - T_1$

- Ideal: S=p and E=1. Superlinear speedup can occur from cache effects, but is not the normal expectation.
- Always identify the baseline: T_1 may mean the best serial code, not one worker running the parallel code.

Quick check: speedup above p or efficiency above 1 needs an explanation; it is not automatically impossible.

2. AMDAHL VS. GUSTAFSON

Model	Formula	Use when
Amdahl	S = 1 / [s + (1-s)/p]	Fixed total problem size
Limit	S(∞) = 1/s	Upper bound from serial fraction
Inverse	s = (1/S - 1/p)/(1 - 1/p)	Infer serial fraction
Gustafson	S = p - (p-1)s	Problem grows with p

Example: s=0.10, p=8. Amdahl gives S=1/[0.10+0.90/8]=4.71, not 8. The serial 10% controls the ceiling: 10x.

Exam signal: 'fixed workload' -> Amdahl. 'Scaled workload' or 'constant runtime as processors increase' -> Gustafson.

3. ROOFLINE MODEL

Operational intensity: I = FLOPs / bytes moved

Attainable $P <= \min(P_{peak}, B \times I)$

Ridge point: $I^* = P_{peak} / B$

If	Bound	Best first move
I < I*	Memory bandwidth	Reuse data; reduce bytes
I > I*	Compute	Improve arithmetic throughput

- Unit shortcut: GB/s x FLOP/byte = GFLOP/s when decimal units match.
- Raising I stops helping after the compute roof is reached.

4. NUMERICAL QUESTION ROUTINE

1. Write the requested quantity and units. 2. Select the model from the wording. 3. Substitute symbolically. 4. Compute. 5. Sanity-check limits and units.

5. DESIGNING A PARALLEL ALGORITHM

Question	What to inspect
Dependencies	Can tasks run now, or must data arrive first?
Granularity	Is each task large enough to amortize overhead?
Concurrency	Are there enough ready tasks for all workers?
Balance	Does each worker receive similar useful work?
Locality	Are reused/nearby data kept close to the worker?

- Fine-grained: more parallelism, more scheduling/synchronization cost.
- Coarse-grained: lower overhead, but may expose too little parallelism or imbalance.
- Static scheduling: low overhead, good for predictable work. Dynamic: better balance for irregular work, higher overhead.

6. COLLECTIVES - KNOW THE DATA MOVEMENT

Collective	Before -> after
Broadcast	One root value -> same value at every rank
Scatter	Root array chunks -> one chunk per rank
Gather	One chunk per rank -> ordered array at root
Allgather	One chunk per rank -> full array at every rank
Reduce	Values -> combined result at root
Allreduce	Values -> combined result at every rank
Barrier	No rank/thread proceeds until all arrive

Do not confuse Allgather with Allreduce: concatenation/collection versus arithmetic or logical combination.

7. OPENMP AND MPI PATTERNS

```
#pragma omp parallel for reduction(+:sum)
for (int i=0; i<n; ++i) sum += a[i];
```

- OpenMP threads share an address space. Mark loop-private temporaries private; use reduction for associative accumulations.
- MPI ranks have separate address spaces. Data moves by explicit messages or collectives.

Row-partitioned matrix-vector multiply: broadcast x -> each rank computes local rows of y -> gather y at root. If everyone needs y, use Allgather instead.

- Blocking send/receive can deadlock if every rank waits. Use safe ordering, Sendrecv, or nonblocking calls.

8. DEPENDENCY CHECK

For every read, ask: who wrote this value, and is that write guaranteed visible before the read? This single question catches races in CPU threads, MPI protocols, and CUDA blocks.

9. CPU PARALLELISM TAXONOMY

Level	Mechanism	Main idea
ILP	Superscalar, out-of-order	One thread; multiple ready instructions
DLP	SIMD/vector	One instruction; many data lanes
TLP	SMT, multicore	Multiple threads in flight
Process	MPI/distributed	Separate memories; messages

- Superscalar: issue multiple independent instructions per cycle when resources and dependencies allow.
- Out-of-order: dynamically execute ready instructions while preserving the program's architectural result.
- SIMD: one instruction controls multiple lanes, reducing instruction fetch/decode work.

SMT shares one core's execution resources. Multicore provides separate cores. SIMD uses lanes inside one instruction.

10. COHERENCE, CONSISTENCY, FALSE SHARING

Term	Question it answers
Coherence	For one memory location, do cores eventually agree on the latest write?
Consistency	What ordering of reads/writes across locations may programs observe?
False sharing	Are independent variables on the same cache line causing invalidations?

False-sharing pattern: thread 0 writes counter[0], thread 1 writes counter[1]. Different words, but the same cache line repeatedly moves between cores.

Fixes: pad/align per-thread state to separate cache lines; accumulate privately and combine later; partition data by contiguous chunks.

Coherence can be correct while performance is terrible. False sharing is mainly a performance problem, not a logical data race.

11. SHARED-MEMORY CORRECTNESS

- A data race occurs when concurrent accesses touch the same location, at least one is a write, and ordering is not guaranteed.
- A barrier orders phases; it does not make a conflicting update atomic.
- An atomic protects one update but may serialize heavy contention.
- A reduction uses private partial results, then combines them; usually better than one hot atomic variable.

Correctness recipe: private where possible -> reduction for associative combine -> atomic/lock for truly shared updates -> barrier only when every participant must finish a phase.

12. FAST TRUE/FALSE FILTER

Absolute words are suspicious: always, never, no cost. Check the limiting case. A barrier is collective. More threads can hurt. More operational intensity does not exceed the compute roof.

COMPUTING 2 - PARALLEL COMPUTING

GPU execution - warps and divergence - memory spaces - coalescing - CUDA indexing - synchronization

13. CUDA EXECUTION HIERARCHY

Level	Meaning / rule
Grid	All blocks launched by one kernel
Block	Thread group assigned to one SM; can share shared memory and use __syncthreads()
Warp	Hardware execution group; NVIDIA warp = 32 threads
Thread	Own logical ID, registers, and sequential instruction stream

global thread i = blockIdx.x x blockDim.x + threadIdx.x

number of blocks = ceil(n / B) = (n + B - 1) / B

```
kernel<<<(n+B-1)/B, B>>>(...);
if (i < n) { /* valid element */ }
```

Blocks may run in any order. Never assume block 0 finishes before block 1.

14. WHAT A WARP REALLY MEANS

Each thread has its own data and logical state, but the GPU issues one instruction for the warp. Active lanes execute that instruction on their own operands.

If lanes choose different branches, the warp normally runs each path with a different active-lane mask. This is divergence; inactive lanes waste issue capacity.

```
if (threadIdx.x % 2 == 0) A();
else B();   // likely two serialized paths
```

- No divergence when every active lane takes the same path.
- Divergence is most costly when branch choices differ within the same warp. Different warps choosing different paths is fine.

Software chooses blocks/threads. Hardware forms and schedules warps; you do not launch a warp directly.

15. ARCHITECTURE FACTS

Machine	Execution group / note
NVIDIA	Warp of 32 threads; SIMT execution
AMD GCN	Historically a 64-lane wavefront
Arm Valhall	Hardware dynamic scheduling; superscalar execution

CPU design spends more hardware on latency and control; GPU design devotes more resources to throughput and hides latency with many ready warps.

16. CUDA MEMORY SPACES

Space	Scope / use	Exam cue
Registers	Per thread; fastest	Private scalars; spilling is costly
Shared	Per block; on chip	Explicit load/store; block cooperation
Global	Device-wide DRAM	High latency; access pattern matters
Constant	Read-only, cached	Good when lanes read same value
Texture / read-only	Cached read path	Read-only spatial locality
Unified	Managed address space	cudaMallocManaged; migration may occur

Shared memory is not automatically faster overall: copying, barriers, bank conflicts, and lack of reuse can erase the benefit.

17. COALESCING - THE EXAM DEFINITION

A warp is coalesced when its active lanes access addresses that can be served by a small number of aligned memory transactions. For contiguous 4-byte values, the ideal pattern is usually lane t -> base+t.

```
lane:      0   1   2   3   ...
address:   A0  A1  A2  A3  ...   // unit stride
```

Check addresses across lanes for one instruction, not whether each thread accesses adjacent values over time.

Index across lanes	Pattern	Likely result
base + t	0,1,2,3,...	Coalesced / few transactions
base + 2t	0,2,4,6,...	Stride 2 / more segments
base + t x W	Column of row-major array	Large stride / poor coalescing

Diagnostic: substitute threadIdx.x = 0,1,2,3. If the first accesses are consecutive, the mapping is coalescing-friendly.

18. TWO OUTPUTS PER THREAD

To preserve coalescing, give each warp two contiguous sections, not two adjacent elements per thread:

```
int i = 2*blockIdx.x*blockDim.x + threadIdx.x;
int j = i + blockDim.x;
if (i < n) C[i] = A[i] + B[i];
if (j < n) C[j] = A[j] + B[j];
```

For the first instruction, lanes access i=base+0, base+1, ... Therefore the correct initialization in the common multiple-choice question is (a). Formula (b) gives 0,2,4,6,... across lanes.

Adjacent outputs inside each thread can still be strided across the warp. Warp-level adjacency is what matters.

19. INDEXING PATTERNS TO MEMORIZE

1D vector kernel

```
int i = blockIdx.x*blockDim.x + threadIdx.x;
if (i < n) C[i] = A[i] + B[i];
```

2D row-major matrix

```
int col = blockIdx.x*blockDim.x + threadIdx.x;
int row = blockIdx.y*blockDim.y + threadIdx.y;
if (row < H && col < W) {
  int idx = row*W + col;
  C[idx] = A[idx] + B[idx];
}
```

- Map threadIdx.x to columns so neighboring lanes touch neighboring row-major elements.
- Guard every computed index. A launch usually rounds up, creating extra threads.

Launch geometry: dim3 block(BX,BY); dim3 grid((W+BX-1)/BX, (H+BY-1)/BY);

20. SYNCHRONIZATION AND TRANSPOSE

A common transpose pattern is global load -> shared tile -> barrier -> swapped read -> global store. The barrier prevents a thread from reading tile data before another thread writes it.

```
tile[ty][tx] = in[row*W + col];
__syncthreads();
out[outRow*H + outCol] = tile[tx][ty];
```

Rule	Consequence
Block-local	__syncthreads() does not synchronize blocks
Collective	All threads in the block must reach it consistently
No ordinary grid barrier	Split phases into separate kernel launches
Visibility + ordering	Earlier shared/global writes become usable in the next block phase

A barrier inside a branch taken by only some block threads can deadlock or be undefined. Move it outside non-uniform control flow.

21. CUDA CODE QUESTION CHECKLIST

1. Expand IDs for lanes 0-3. 2. Verify index bounds. 3. List addresses touched by one warp instruction. 4. Find cross-thread producer/consumer pairs. 5. Place a block barrier only if every thread reaches it. 6. Never assume block order.

Common traps: B[I] typo/case mismatch; missing second bound check; row/column swapped; stride-2 mapping; confusing a logical thread with a warp; calling a barrier an atomic operation.

22. LAST-MINUTE FACT BANK

Warp divergence has a cost. Shared memory uses explicit instructions. cudaMallocManaged creates Unified Memory. Vector instructions reduce instruction fetch/decode demand. A GPU core is simpler because throughput, not single-thread latency, dominates the design.